

Converting the Project into an LLM-Assisted Precision 3D Modeling Tool

Vision

Transform the current edu3d DSL visualizer + compiler stack into a **precision 3D modeling tool** where:

- Users describe models in natural language or directly edit DSL
- An LLM interprets intent and generates well-formed `.dsl` files
- The DSL compiler validates and compiles models with physical precision
- The Three.js / wgpu renderer produces interactive 3D visuals
- Models are saved and exchanged exclusively as `.dsl` files (the single source of truth)

Current State Summary

Layer	What Exists
<code>dsl/</code>	Full Rust DSL compiler: lexer → parser → 5-pass validator → IR lowering
<code>compiler/renderer/</code>	Three.js (WASM) + native wgpu backends, bridge, frame sync
<code>web_service/</code>	Minimal FastAPI server serving a Three.js canvas; mock WebSocket snapshot loop
<code>mathlib/</code>	Geometry primitives (Point, Plane, Vector, Scalar) in Python

The DSL grammar (`grammar.md`) already supports entities, transforms, geometry, constraints, motions, materials, and timelines — it is the right foundation for a modeling tool.

1. DSL Layer Changes (`dsl/`)

1.1 Add `modeling` Domain to Grammar (`grammar.md`)

The grammar already defines `domain_type`. Add a `modeling` domain alongside `physics`, `chemistry`, etc.

Change: In `grammar.md` section 3:

```
diff
```

```
- domain_type ::= "physics" | "chemistry" | "mathematics" | "robotics" | "general"
+ domain_type ::= "physics" | "chemistry" | "mathematics" | "robotics" | "modeling" | "general"
```

New domain-specific components for **modeling**:

```
solid      { primitive, dimensions, fillet_radius? }
boolean_op { operation: "union"|"subtract"|"intersect", target, tool }
sketch     { plane, closed, points[] }
extrude    { sketch_ref, depth, taper_angle? }
revolve    { sketch_ref, axis, angle }
loft       { profiles[], guide_curves[]? }
sweep      { profile, path }
shell      { thickness, open_faces[]? }
chamfer    { edges[], distance }
fillet     { edges[], radius }
thread     { face, pitch, handedness }
annotation { type: "dimension"|"note"|"gd&t", target, value }
```

New constraints for modeling:

```
coincident { entity_a, entity_b }
parallel   { entity_a, entity_b }
perpendicular { entity_a, entity_b }
tangent    { entity_a, entity_b }
distance   { entity_a, entity_b, value, unit }
angle      { entity_a, entity_b, value }
symmetric  { entity_a, entity_b, axis }
```

1.2 Precision Annotation in DSL ((**grammar.md**) + (**validator/units.rs**))

Add a **precision** block to the scene header and enforce it during validation:

```
scene {
  name: "bracket_v1"
  version: 1
  ir_version: "2.0.0"
  unit_system: SI
  domain: modeling
  precision: 0.001 # Global tolerance in scene units (mm if unit_system = SI)
}
```

Change in `validator/units.rs`: Add a `validate_precision()` pass that:

- Ensures all dimension values have enough decimal places to satisfy precision
- Warns when two objects are closer than `precision` without an explicit `coincident` constraint
- Emits `E401` for precision underspecification

1.3 New Error Codes (`errors.rs`)

Add error range `E1000–E1099` for modeling-specific errors:

E1001: Invalid boolean operation (tool and target must be closed solids)
E1002: Sketch not closed (cannot extrude open sketch)
E1003: Constraint conflict detected
E1004: Dimension below precision threshold
E1005: Feature depends on deleted geometry

1.4 New Validator Pass: `validator/geometry.rs`

Add **Pass 6** — Geometric Integrity:

- Verifies boolean operations are between closed manifold solids
- Verifies sketch profiles are closed when used in extrude/revolve
- Detects self-intersecting loft profiles
- Checks fillet/chamfer radii don't exceed edge length

1.5 IR Output — Model Geometry IR (`lower_to_ir.rs`)

Extend `IrScene` with a `modeling_tree: Option<Vec<IrFeature>>` field representing the ordered feature tree (parametric history). Each `IrFeature` wraps one DSL entity with its resolved operation type, inputs, and parameters.

2. DSL File Format Changes

The `.dsl` file becomes the **save format** for models. Each saved model is a self-contained, human-readable, version-controlled `.dsl` file.

2.1 Model File Header Convention

dsl

```
scene {
  name: "bracket_assembly_v3"
  version: 3
  ir_version: "2.0.0"
  unit_system: SI      # mm as base length
  domain: modeling
  precision: 0.01      # 0.01 mm tolerance
  author: "user@example.com"
  created: "2026-02-26"
}
```

2.2 Feature Tree Ordering

DSL section order becomes the **parametric feature tree** (like a CAD history tree). Features listed later can reference earlier entities. This is already enforced by the parser's "no forward references" rule — no change needed, just document this as the modeling idiom.

2.3 File Save / Load API

Change in `web_service/app/main.py` — add endpoints:

```
python

POST /api/model/save      # Body: { name, dsl_source } → saves .dsl file
GET  /api/model/load/{name} # Returns raw .dsl source text
GET  /api/model/list      # Returns list of saved model filenames
POST /api/model/compile    # Body: { dsl_source } → returns IR JSON or errors
```

The server stores models as `.dsl` files on disk (a `models/` directory). No binary format, no database — the DSL file IS the save format.

3. LLM Integration Layer (New)

3.1 Role of the LLM — Description, Not Code

The LLM's job is **not** to write DSL. Its job is to act as a design consultant: given a user's natural language request, it produces a rich structured description of what the model should look like — shapes, dimensions, spatial relationships, materials, and features — written as a human-readable design document, not as code.

A separate, deterministic translation layer then reads that description and emits the DSL. This separation means:

- The LLM stays in its domain of strength (understanding intent, describing form)

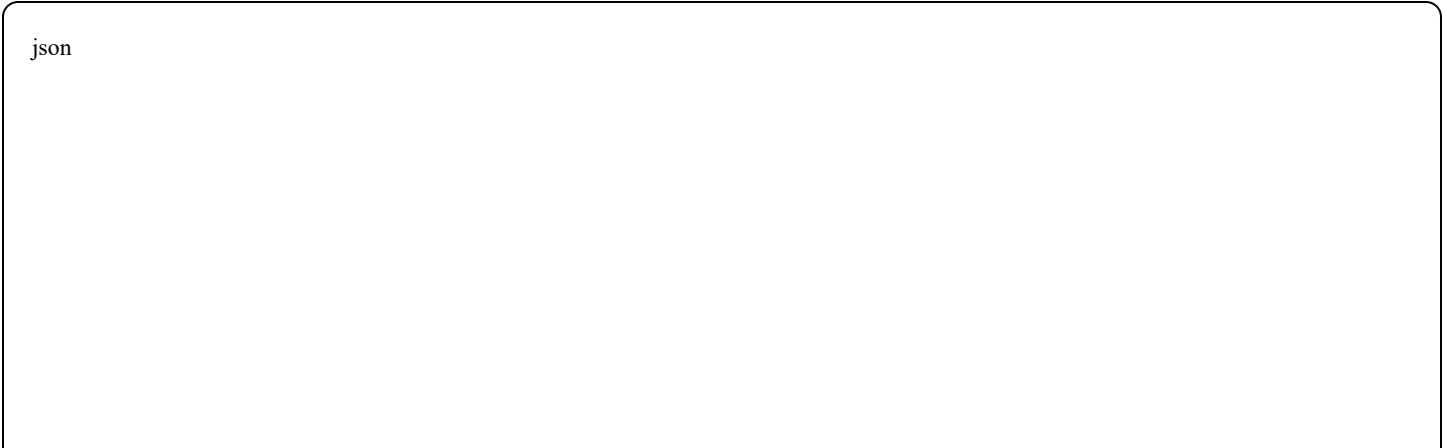
- DSL generation stays deterministic and auditable
- LLM errors appear as description ambiguities, not syntax failures
- Users can read and correct the description before it becomes geometry



3.2 What the LLM Produces — The Model Description Format

The LLM returns a structured JSON description (not DSL, not prose). This is the intermediate representation between user intent and machine-executable DSL. It is human-readable, editable by the user in the UI, and the source of truth before DSL is generated.

Model Description Schema:



```
{
  "name": "bracket_assembly",
  "unit": "mm",
  "precision": 0.01,
  "summary": "A rectangular mounting bracket with four bolt holes and filleted top edges.",
  "shapes": [
    {
      "id": "body",
      "type": "box",
      "label": "Main body",
      "dimensions": { "width": 50, "height": 30, "depth": 10 },
      "position": [0, 0, 0],
      "material": "steel"
    },
    {
      "id": "hole_fl",
      "type": "cylinder",
      "label": "Front-left bolt hole",
      "dimensions": { "radius": 3.1, "depth": 10 },
      "position": [-18, -10, 0],
      "operation": "subtract",
      "target": "body"
    }
  ],
  "features": [
    {
      "type": "fillet",
      "label": "Top edge fillets",
      "target_edges": ["body_top_front", "body_top_back"],
      "radius": 4
    }
  ],
  "constraints": [
    {
      "type": "symmetric",
      "label": "Bolt holes symmetric about Y axis",
      "entities": ["hole_fl", "hole_fr"],
      "axis": [0, 1, 0]
    }
  ],
  "notes": "All bolt holes are M6 clearance (3.1mm radius). Fillets apply to the four top longitudinal edges only."
}
```

This description lives in the UI as an editable panel. The user can tweak values, rename shapes, or change dimensions directly in this JSON before triggering DSL generation.

3.3 New Module: `web_service/app/llm_bridge.py`

This module manages the LLM call and receives back a Model Description.

Key function:

```
python

async def generate_description(
    user_prompt: str,
    current_description: dict | None, # existing description if refining
    unit_system: str,
    precision: float,
) -> dict: # returns Model Description JSON
```

System prompt template (stored in `web_service/app/prompts/description_system.txt`):

You are a precision 3D modeling assistant. When the user describes a model, you produce a structured JSON description of what the model should look like.

Rules:

- Output ONLY valid JSON. No prose, no markdown, no explanation outside the JSON.
- Every shape must have an id, type, label, dimensions, and position.
- Dimensions must be explicit numbers in {unit_system} units.
- Positions are the center of the shape in 3D space.
- Boolean operations (subtract, union, intersect) reference a target shape by id.
- Features (fillets, chamfers, threads) reference target edges or faces by id.
- Constraints describe spatial relationships between shapes.
- The "notes" field captures anything that cannot be expressed structurally.
- If the user says "like a standard M6 bolt hole", use 3.1mm radius.
- If dimensions are ambiguous, make a reasonable assumption and note it.

Unit system: {unit_system}

Precision: {precision} (smallest meaningful dimension difference)

If the user is refining an existing description, return the full updated description preserving all shapes and features not mentioned in the refinement request.

Temperature is set to **0.3** — enough for natural language understanding, low enough for consistent structured output.

3.4 New Module: `web_service/app/description_to_dsl.py`

This is the deterministic translation layer. It takes a validated Model Description JSON and emits a `.dsl` file. No LLM is involved here — it is a straightforward mapping.

```
python

def description_to_dsl(description: dict) -> str:
    """
    Deterministically translates a Model Description into a DSL source string.
    Rules:
    - Each shape in description["shapes"] → one entity block
    - Boolean operations → constraint blocks
    - Features → additional entity blocks (fillet entity, etc.)
    - Constraints → constraint blocks
    - Material names → material blocks
    """
```

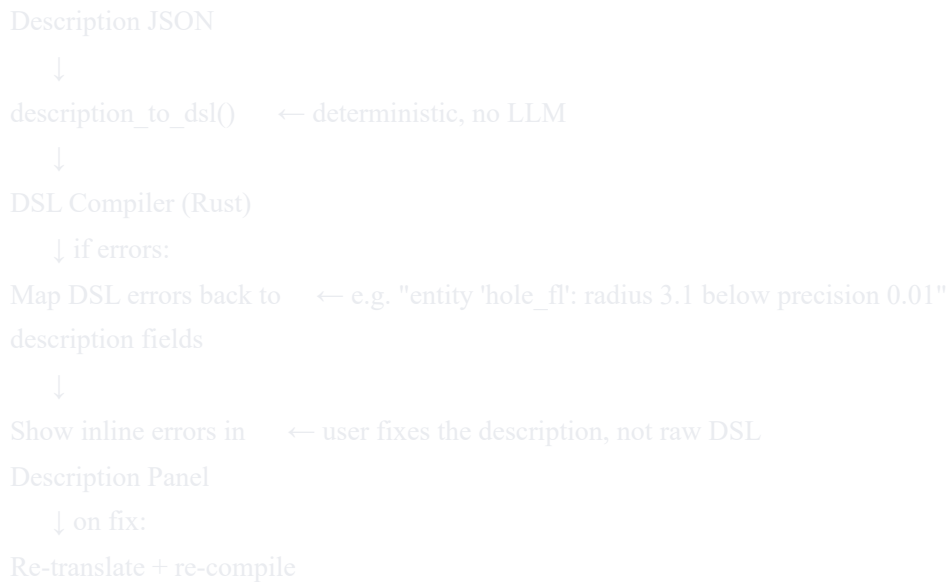
Mapping rules:

Description field	DSL construct
<code>shape.type = "box"</code>	<code>entity { kind: solid; components { geometry { primitive: box; dimensions: [...] } } }</code>
<code>shape.operation = "subtract"</code>	<code>constraint { type: boolean_subtract; tool: shape_id; target: target_id }</code>
<code>feature.type = "fillet"</code>	<code>entity { kind: fillet; components { fillet { edges: [...]; radius: N } } }</code>
<code>constraint.type = "symmetric"</code>	<code>constraint { type: symmetric; entity_a: ...; entity_b: ...; axis: [...] }</code>
<code>shape.material</code>	<code>material</code> block in the <code>materials</code> section

Because the description schema is well-defined, this translation is fully deterministic and testable with unit tests — no ambiguity, no LLM involvement.

3.5 Validation Loop

The translated DSL is compiled immediately. If it fails, the error is reported against the **description** (not the DSL directly), so the user sees something actionable:



There is no LLM retry loop here. Compile errors are the result of a bad description translation (a code bug), not LLM output. The LLM only generates the description once per user prompt; the compile loop is entirely between the description and the DSL translator.

3.6 New API Endpoints

Change in `web_service/app/main.py`:

```
python
```

POST /api/llm/describe

- # Body: { prompt: str, current_description?: dict, unit_system?: str, precision?: float }
- # Returns: { description: ModelDescription }
- # Behavior: calls LLM, returns structured Model Description JSON
- # The LLM never sees the DSL compiler or grammar spec

POST /api/llm/refine

- # Body: { prompt: str, current_description: dict }
- # Returns: { description: ModelDescription, changed_fields: [str] }
- # Behavior: LLM updates specific shapes/features in an existing description
- # Returns list of changed field paths so UI can highlight what changed

POST /api/description/to_dsl

- # Body: { description: ModelDescription }
- # Returns: { dsl: str, errors: [] }
- # Behavior: deterministic translation only, no LLM involved
- # Fast — this is called on every description edit

POST /api/llm/explain

- # Body: { description: ModelDescription }
- # Returns: { explanation: str }
- # Behavior: LLM reads a Model Description and returns a plain English summary
- # Useful when a description was produced by GUI or DSL script, not LLM

3.7 Three Authoring Modes — How They All Connect

All three ways of building a model converge on the same DSL → IR → render pipeline. The LLM path adds the description as an intermediate artifact.

```
Mode A: LLM Assisted
User types natural language
↓ POST /api/llm/describe
LLM produces Model Description JSON
↓ shown in Description Panel (editable)
User tweaks description if needed
↓ POST /api/description/to_dsl (auto)
DSL generated deterministically
```

```
Mode B: GUI / Direct Modeling
User adds shapes, applies operations via toolbar
↓ each action updates a live scene graph
Scene graph serialized to Model Description JSON
↓ POST /api/description/to_dsl (auto)
DSL generated deterministically
```

```
Mode C: DSL Script
User writes .dsl directly in the code editor
↓ POST /api/model/compile (on save / debounce)
DSL compiled directly — no description involved
Reverse-parse DSL → Description for Properties panel
```

All three modes → DSL Compiler (Rust) → IrScene → Three.js Renderer

In Mode A and B the Model Description JSON is always visible to the user as an editable intermediate. In Mode C it is optionally reverse-generated for the properties panel but is not authoritative.

3.8 LLM Model and Prompt Engineering Notes

Use `claude-sonnet-4-6` via the Anthropic API.

The LLM **never receives** `grammar.md` or any DSL specification. It has no knowledge of how the DSL works. Its only job is to describe geometry. This keeps the system prompt small, focused, and independent of DSL version changes.

Key prompt decisions:

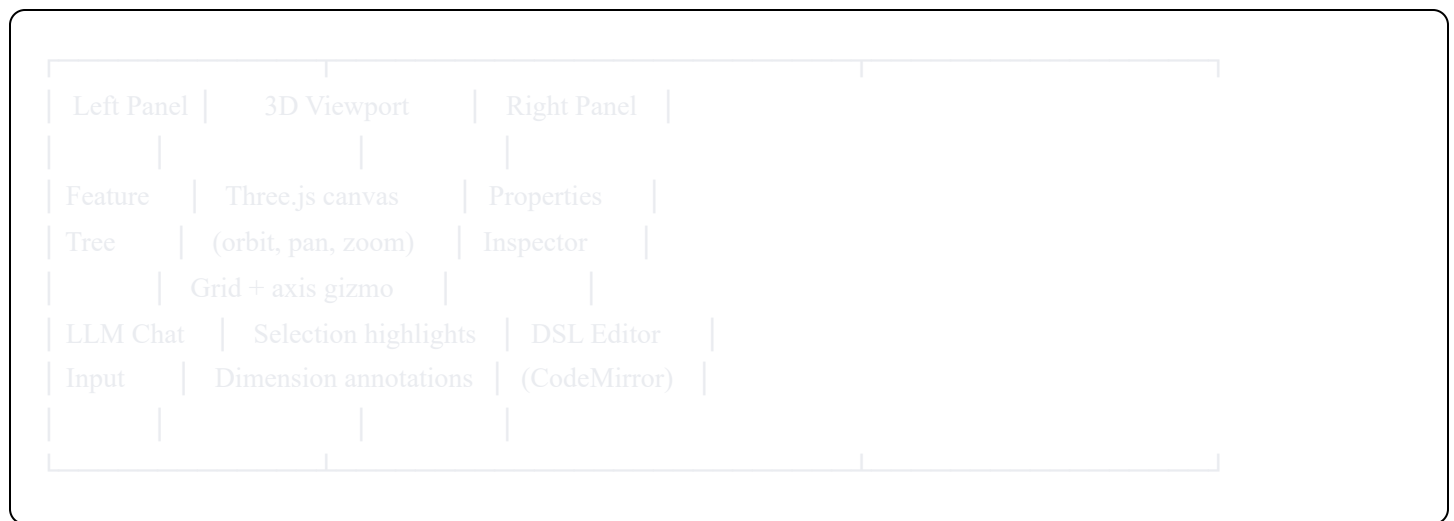
- Include 2–3 example (user prompt → description JSON) pairs as few-shot examples in the system prompt
- Always specify units and precision in the system prompt so the LLM uses concrete numbers
- For refinement calls, send the full current description so the LLM can preserve unchanged parts
- Parse the response as JSON with strict validation; if the JSON is malformed, retry once with a correction note ("your previous response was not valid JSON, please try again")

4. Frontend / Renderer Changes (`web_service/app/static/` + `compiler/renderer/`)

4.1 Rebuild `index.html` as a Modeling UI

The current `index.html` is a minimal debug viewer. Replace it with a full modeling interface:

Layout (three-panel):



Left Panel:

- Feature Tree showing DSL entities in order
- Clicking a feature highlights it in the viewport and scrolls the DSL editor
- LLM prompt input at the bottom ("Describe a change...")

Center Viewport (Three.js):

- Orbit controls (already present, needs OrbitControls import from CDN)
- World grid (THREE.GridHelper)
- Axis gizmo (corner overlay)
- Selection: click object → highlight + show properties
- Dimension annotation overlays (CSS2DRenderer for labels)

- Measurement display (edge lengths, angles)

Right Panel:

- Properties inspector: shows selected entity's component values (editable)
- Inline DSL editor (CodeMirror with DSL syntax highlighting)
- Editing DSL directly → triggers recompile on Ctrl+S or after 800ms debounce
- Compile errors shown inline with red underlines

4.2 Real-Time Compile Feedback

Change in `index.html` JavaScript:

```
javascript

// Replace the mock snapshot loop with:
async function compileDsl(source) {
  const res = await fetch('/api/model/compile', {
    method: 'POST',
    body: JSON.stringify({ dsl_source: source }),
    headers: { 'Content-Type': 'application/json' }
  });
  const { ir, errors } = await res.json();
  if (errors.length) showErrors(errors);
  else updateScene(ir); // apply IR to Three.js scene
}
```

4.3 `updateScene(ir)` — IR → Three.js Mapper

New JavaScript function that reads the compiled `IrScene` JSON and creates/updates Three.js meshes. Maps `IrEntity.components.geometry.primitive` to Three.js geometry classes:

```
box    → THREE.BoxGeometry(w, h, d)
sphere → THREE.SphereGeometry(r, 32, 32)
cylinder → THREE.CylinderGeometry(r, r, h, 32)
cone    → THREE.ConeGeometry(r, h, 32)
torus    → THREE.TorusGeometry(R, r, 16, 100)
plane    → THREE.PlaneGeometry(w, h)
```

Boolean ops are executed client-side using `three-bvh-csg` (imported from CDN).

4.4 Precision Grid and Snapping

Add to Three.js viewport:

- Grid spacing matches scene `precision` value
- Snap-to-grid toggle (keyboard shortcut `G`)
- Snap-to-vertex, snap-to-edge toggles
- Measurement tool: click two points → shows distance with unit label

4.5 Renderer Backend: Add Measurement Geometry (`compiler/renderer/`)

Change in `compiler/renderer/backend/traits.rs` — add to `RenderBackend` trait:

```
rust
```

```
fn create_annotation(&mut self, annotation: RenderAnnotation) -> RenderResult<u64>;  
fn update_annotation(&mut self, id: u64, annotation: RenderAnnotation) -> RenderResult<>;
```

```
rust
```

```
pub enum RenderAnnotation {  
    LinearDimension { start: [f32;3], end: [f32;3], label: String },  
    AngleDimension { vertex: [f32;3], ray1: [f32;3], ray2: [f32;3], label: String },  
    Note           { position: [f32;3], text: String },  
}
```

Implement in `three_js.rs` using `CSS2DObject` for the label overlays.

5. WebSocket Snapshot — Replace Mock with Real

Change in `web_service/app/main.py`:

Remove the mock tick loop. Replace `snapshot` with the actual `IrScene` JSON produced by the compiler. The WebSocket now pushes snapshot updates only when a model is recompiled (event-driven, not polling):

```
python
```

```

# Global state
_current_ir: dict = {}
_ws_clients: list[WebSocket] = []

async def broadcast_ir():
    for ws in _ws_clients:
        await ws.send_text(json.dumps(_current_ir))

@app.post("/api/model/compile")
async def compile_model(body: CompileRequest):
    result = run_dsl_compiler(body.dsl_source)
    if result.ok:
        _current_ir = result.ir
        await broadcast_ir()
    return result

```

6. Precision Measurement Module (New Python, Optional Rust Port Later)

New file: `web_service/app/precision.py`

Wraps `mathlib/geometry/` to offer a measurement API used both server-side (for validation feedback) and callable from the LLM bridge:

```

python

def measure_distance(entity_a_ir, entity_b_ir) -> float
def measure_angle(entity_a_ir, entity_b_ir) -> float
def check_coincident(entity_a_ir, entity_b_ir, tolerance: float) -> bool
def compute_bounding_box(ir_scene) -> BoundingBox
def compute_volume(entity_ir) -> float # for closed solids only

```

Results are included in the compile response so the UI can display them in the properties panel.

7. New Project Structure

```

project/
|
|— dsl/           # Rust DSL compiler (mostly unchanged)
|  |— validator/

```

```

├── geometry.rs      # NEW: Pass 6 — geometric integrity
├── ...
├── compiler/renderer/ # Rust renderer backends (minor additions)
│   ├── backend/
│   │   └── traits.rs # Add RenderAnnotation type + trait methods
├── mathlib/         # Python geometry math (unchanged)
├── web_service/
│   ├── app/
│   │   ├── main.py      # CHANGED: new endpoints, event-driven WS
│   │   ├── llm_bridge.py # NEW: user prompt → Model Description JSON (LLM only)
│   │   ├── description_to_dsl.py # NEW: Model Description → DSL (deterministic, no LLM)
│   │   ├── precision.py  # NEW: measurement API
│   │   ├── prompts/
│   │   │   └── description_system.txt # NEW: LLM system prompt (geometry description only)
│   │   └── static/
│   │       ├── index.html      # REPLACED: full three-panel modeling UI
│   │       ├── dsl-editor.js    # NEW: CodeMirror DSL editor (Mode C)
│   │       ├── description-panel.js # NEW: editable Model Description JSON panel (Mode A/B)
│   │       ├── scene-manager.js # NEW: IR → Three.js mapper
│   │       ├── gui-tools.js     # NEW: shape toolbar, transform gizmos (Mode B)
│   │       └── feature-tree.js  # NEW: left panel feature tree
├── models/           # NEW: saved .dsl model files (user workspace)
│   └── examples/
│       └── bracket.dsl

```

8. Workflow Summary (End to End)

Mode A — LLM Assisted

User types: "Create a 50mm × 30mm × 10mm rectangular bracket with
4mm fillets on the top edges and an M6 bolt hole centered on top"

↓

POST /api/llm/describe

- LLM receives: user prompt + unit system + precision
- LLM has NO knowledge of DSL grammar
- LLM returns: Model Description JSON

```
{
```

```
shapes: [ {id:"body", type:"box", dimensions:{w:50,h:30,d:10}},  
          {id:"hole", type:"cylinder", radius:3.1, operation:"subtract"} ],  
features: [ {type:"fillet", edges:["body_top_*"], radius:4} ]  
}
```



Description Panel shown to user (editable JSON / friendly form)

- User can adjust dimensions, rename shapes, change materials
- Every edit triggers re-translation automatically



POST /api/description/to_dsl (deterministic, no LLM)

- description_to_dsl() maps each shape → entity block
- boolean ops → constraint blocks, fillets → entity blocks
- Returns clean DSL source



DSL Compiler (Rust)

- Lexer → Parser → 6-pass Validator → IR Lowering
- Errors mapped back to description fields, shown inline
- On success: IrScene JSON emitted



Frontend (Three.js)

- updateScene(ir): BoxGeometry + CylinderGeometry boolean subtract + fillet
- Feature Tree: [body, hole, fillet_top_edges]
- Description Panel and DSL Editor both in sync



User saves → bracket.dsl written to disk

Mode B — GUI Direct Modeling

User clicks "Add Box" → sets dimensions in properties panel



Scene graph updated in memory

- Shape added to Description JSON internally



POST /api/description/to_dsl (auto, same path as Mode A)



DSL Compiler → IR → Three.js render (live update)

Mode C — DSL Script

User writes DSL directly in the code editor panel



POST /api/model/compile (on Ctrl+S or 800ms debounce)

- No description translation involved
- Compile errors shown as inline underlines in editor



IrScene → Three.js render

- Reverse-parse IR → Description JSON for Properties Panel display



User saves → .dsl file written directly

9. Priority Order for Implementation

1. **DSL grammar extension** — add `modeling` domain, new components/constraints (`grammar.md`, `validator/schema.rs`, `validator/library.rs`)
2. `validator/geometry.rs` — Pass 6 for geometric integrity
3. **Web service core endpoints** — save/load/compile/list (`main.py`)
4. `description_to_dsl.py` — deterministic description → DSL translator (no LLM, fully testable)
5. `llm_bridge.py` — LLM → Model Description JSON (description only, no grammar spec in prompt)
6. **Frontend rebuild** — three-panel UI, feature tree, Description Panel, DSL editor, IR→Three.js mapper
7. **GUI modeling tools** — shape toolbar, transform gizmos, boolean operation UI (feeds scene graph → description)
8. **Annotation rendering** — dimension labels in Three.js (CSS2DRenderer)
9. `precision.py` — measurement utilities
10. **Snapping + measurement tools** — viewport interaction polish

10. Dependencies to Add

Package	Where	Purpose
<code>anthropic</code> (Python)	<code>web_service/</code>	LLM API — description generation only
<code>aiofiles</code> (Python)	<code>web_service/</code>	Async .dsl file I/O
<code>jsonschema</code> (Python)	<code>web_service/</code>	Validate Model Description JSON before translation
<code>CodeMirror 6</code> (JS, CDN)	<code>static/</code>	DSL syntax editor (Mode C)
<code>three-bvh-csg</code> (JS, CDN)	<code>static/</code>	Client-side boolean ops for preview

Package	Where	Purpose
<code>CSS2DRenderer</code> (Three.js addon)	<code>static/</code>	Dimension label overlays
<code>OrbitControls</code> (Three.js addon)	<code>static/</code>	Viewport navigation
<code>TransformControls</code> (Three.js addon)	<code>static/</code>	GUI move/rotate/scale gizmos (Mode B)